Python Activity-Based Learning : 10 Case Studies

NAME : SK.NAZIYA

ROLL NO : 26001621

BRANCH : AIML

DEPARTMENT : CSE

CONTENTS

1.Bus passenger occupancy simulation

2. Hackathon Team Formation and Ranking

3. Food Waste Monitoring System

4. Multi-Branch Expense Matrix Analysis

5. Tuple-Based Transaction Audit

6.Student Group Formation with Score Balancing

7. Smart Canteen Order and Stock Management

8. Library Borrowing and Fine Reconciliation

9. Campus Transport Allocation

10. Manufacturing Quality Control Matrix

51.Bus Passenger Occupancy Simulation

1.Problem statement

Process passenger boarding and alighting at multiple bus stops, calculate the passenger count after each stop, and flag any capacity violation.

2.Objectives

• Track passenger movement at each stop.

• Calculate the number of passengers on the bus after boarding and alighting.

• Warn when passenger count exceeds the bus capacity.

3.Assumptions

• Initial passenger count is zero.

• Boarding and alighting counts are non-negative integers.

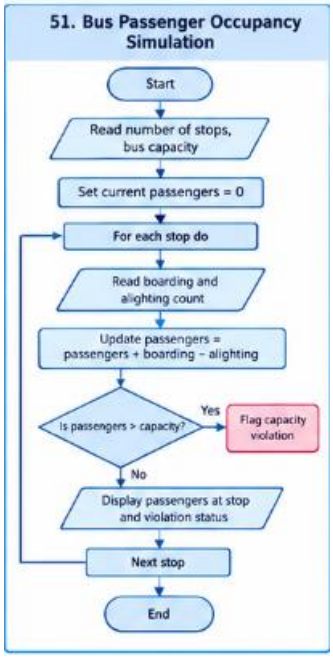
• Passengers cannot alight if they are not on the bus; invalid alighting counts are rejected.

• Capacity is a positive integer.

4.Algorithm

1. Start.
2. Input bus capacity and number of stops.
3. For each stop, input boarding and alighting counts.
4. Validate counts and ensure alighting does not exceed current passengers.
5. Update passengers = passengers - alighting + boarding.
6. Display occupancy and flag capacity violations.
7. Stop.

5.Flow Chart



6.Pseudocode

START

INPUT capacity, stops

```
passengers = 0
FOR each stop
        INPUT boarding, alighting
        IF boarding < 0 OR alighting < 0 OR alighting > passengers
                PRINT invalid counts
         ELSE
                passengers = passengers - alighting + boarding
                PRINT passengers
               IF passengers > capacity: PRINT capacity violation
               ELSE: PRINT within capacity
 STOP
```

7.Python Code

```
capacity  =  int(input("Bus capacity: "))
stops  =  int(input("Number of stops: "))
passengers = 0

if capacity <= 0 or stops < 0:
        print("Invalid capacity or stop count")
 else:
        for i in range(stops):
                print("Stop", i + 1)
                boarding = int(input("Boarding: "))
                alighting = int(input("Alighting: "))

                 if boarding < 0 or alighting < 0 or alighting > passengers:
                        print("Invalid passenger counts")
                        continue
                  passengers = passengers - alighting + boarding
                  print("Passengers:", passengers)
```

```
if passengers > capacity:
        print("CAPACITY VIOLATION")
else:
        print("Within capacity")
```

8.Test Cases

Input/test	Expected Result
Capacity 40; start 0; board 20, alight 0	20 passengers; within capacity
Capacity 30; current 20; board 15, alight 5	30 passengers; within capacity
Current 10; alight 12	Invalid passenger counts

9. Sample Output Screenshot

```
Bus capacity: 30
Number of stops: 2
Stop 1
Boarding: 20
Alighting: 0
Passengers: 20
Within capacity
Stop 2
Boarding: 15
Alighting: 5
Passengers: 30
Within capacity
```

10.Conclusion

The program tracks occupancy across stops, rejects impossible alighting counts, and identifies capacity violations.

52.Hackathon Team Formation and Ranking

1.Problem statement

Analyse team sizes and round-wise scores, reject teams outside the allowed size range, calculate total scores, and rank valid teams. Ties are resolved using the higher Round 1 score.

2.Objectives

• Validate team sizes.

• Aggregate scores from two rounds.

• Rank valid teams in descending order.

• Use Round 1 score as the tie-breaker.

3.Assumptions

• The minimum and maximum team sizes are entered by the user.

• Each team has two non-negative round scores.

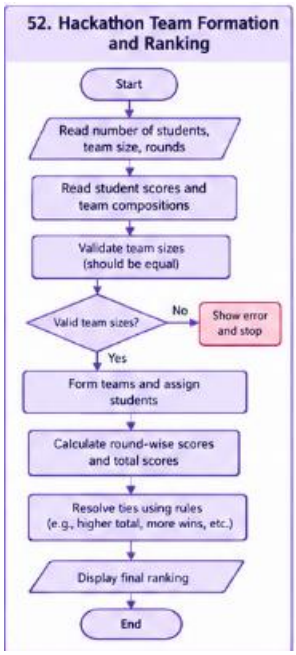
• Only teams with valid sizes are ranked.

• Higher total score ranks first; ties use higher Round 1 score.

4.Algorithms

1. Start.
2. Input number of teams and allowed team-size range.
3. For each team, input name, size, Round 1 score, and Round 2 score.
4. Keep only teams with valid sizes and non-negative scores.
5. Calculate total score.
6. Sort by total score descending, then Round 1 score descending.
7. Display ranking or state that no valid teams exist.
8. Stop

5.Flowchart



6.Pseudocode

```
START
 INPUT n, minimum, maximum
 teams = empty list
 REPEAT n times
        INPUT name, size, round1, round2
        IF minimum <= size <= maximum AND scores >= 0
                ADD (name, round1 + round2, round1) to teams
 SORT teams by total descending, then round1 descending
 DISPLAY rank, name, total
STOP
```

7.Python Code

```
 teams = []
 n = int(input("Number of teams: "))
 minimum = int(input("Minimum team size: "))
 maximum = int(input("Maximum team size: "))


 for i in range(n):
```

```
        print("Team", i + 1)
        name = input("Team name: ")
        size = int(input("Team size: "))
        r1 = int(input("Round 1 score: "))
        r2 = int(input("Round 2 score: "))

        if minimum <= size <= maximum and r1 >= 0 and r2 >= 0:
            teams.append((name, r1 + r2, r1))
        else:
            print("Invalid team or scores; excluded")
teams.sort(key=lambda t: (t[1], t[2]), reverse=True)
 if not teams:
        print("No valid teams")
else:
        for rank, team in enumerate(teams, 1):
            print(rank, team[0], "Total:", team[1]
```

8.Test cases

Input/Test	Expected Result
Team A size 4 scores 80, 70; Team B size 5	Both total 150; Team B ranks first because Round 1 score is higher.
Allowed size 3-5; Team C size 2	Team c is excluded.
No valid teams entered	No valid teams

9.Sample Output Screenshot

```
Enter number of teams: 2
Enter minimum team size: 3
Enter maximum team size: 5

Team 1
Enter team name: Team A
Enter number of students: 80
Invalid team size!

Team 2
Enter team name: Team B
Enter number of students: 2
Invalid team size!

--- Hackathon Team Rankings ---
```

10.conclusion

The program validates team composition, combines round scores, and ranks teams with a defined tie-breaking rule.

53.Food Waste Monitoring System

1. Problem Statement

Compare prepared meals, meals served, and leftovers over several days, calculate the food-wastage percentage, and flag days exceeding an acceptable limit.

2. Objectives

• Calculate leftovers for each day.

• Calculate wastage percentage.

• Identify days above the allowed wastage limit.

3. Assumptions

• Prepared meals must be greater than zero.

• Meals served must be between zero and prepared meals.

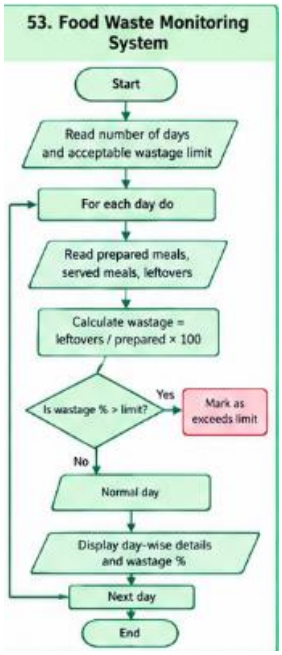
• Leftovers = prepared - served.

• Wastage percentage = leftovers / prepared × 100.

4. Algorithm

1. Start.
2. Input number of days and acceptable wastage percentage.
3. For each day, input prepared and served meals.
4. Validate the values.
5. Calculate leftovers and wastage percentage.
6. Compare percentage with the limit and display status.
7. Stop.

5. Flowchart



6. Pseudocode

```
START
 INPUT days, limit
 FOR each day
            INPUT prepared, served
            IF prepared <= 0 OR served < 0 OR served > prepared
                    PRINT invalid data
             ELSE
                     leftover = prepared - served
                     waste = leftover / prepared * 100
```

PRINT leftover, waste

IF waste > limit: PRINT excessive wastage

ELSE: PRINT acceptable

STOP

7. Python Code

```
days = int(input("Number of days: "))
limit = float(input("Wastage limit (%): "))

for i in range(days):
    print("Day", i + 1)
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))

    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid meal data")
        continue
    leftover = prepared - served
    waste = leftover / prepared * 100
    print("Leftover meals:", leftover)
    print("Wastage:", round(waste, 2), "%")

    if waste > limit:
        print("Excessive wastage")
    else:
        print("Acceptable wastage"
```

8.Test Cases

Input/Test	Expected Result
Prepared 100, served 80, limit 15%	20 leftovers; 20% wastage; excessive.
Prepared 100, served 95, limit 10%	5 leftovers; 5% wastage; acceptable.
Prepared 0	Invalid meal data

9.Sample Output Screenshot

```
Enter number of days: 20
Enter acceptable wastage percentage: 15

Day 1
Enter meals prepared: 100
Enter meals served: 80
Leftover meals: 20
Wastage percentage: 20.0 %
Status: Wastage limit exceeded!

Day 2
Enter meals prepared: 80
Enter meals served: 50
Leftover meals: 30
Wastage percentage: 37.5 %
Status: Wastage limit exceeded!

Day 3
Enter meals prepared: 50
Enter meals served: 40
Leftover meals: 10
Wastage percentage: 20.0 %
Status: Wastage limit exceeded!
```

10.Conclusion

The system calculates daily leftovers and wastage percentages and helps identify days needing waste-reduction action.

54. Multi-Branch Expense Matrix Analysis

1. Problem Statement

Analyse a nested list of monthly expenses for multiple branches and categories, calculate branch totals and averages, and identify the largest expense contributors.

2. Objectives

- Store expenses in a two-dimensional list.
- Calculate each branch's total and average expense.
- Calculate category totals across branches.
- Identify the largest branch and category contributors.

3. Assumptions

• Each row represents a branch and each column represents an expense category.

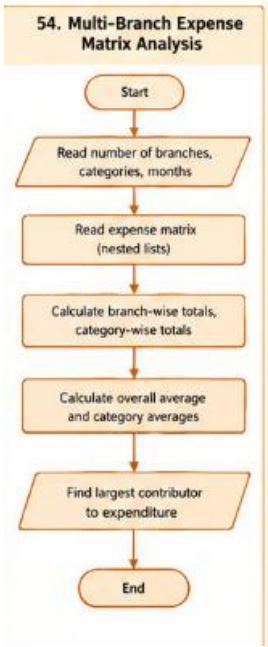
• All expense values are non-negative numbers.

• The sample matrix uses three branches and three categories.

4. Algorithm

1. Start.
2. Create or input the expense matrix.
3. Calculate the total and average for each branch.
4. Calculate the total for each category across all branches.
5. Find the branch with the highest total and the category with the highest total.
6. Display all results.
7. Stop.

5. Flowchart



6. Pseudocode

START

INPUT expense matrix

FOR each branch row

branch_total = SUM(row)

branch_average = branch_total / number_of_categories

```
        PRINT branch_total, branch_average
FOR each category column
        category_total = SUM(values in column)
        PRINT category_total
 FIND maximum branch total and category total
PRINT largest contributors
STOP
```

7. Python Code

```
expenses = [
        [1000, 2000, 1500],
        [3000, 1000, 2500],
        [1200, 1800, 2200]
]
branch_totals = []
for i, branch in enumerate(expenses):
        total = sum(branch)
        average = total / len(branch)
        branch_totals.append(total)
        print("Branch", i + 1, "total:", total)
        print("Branch", i + 1, "average:", round(average, 2))
category_totals = []
for j in range(len(expenses[0])):
        total = sum(row[j] for row in expenses)
        category_totals.append(total)
        print("Category", j + 1, "total:", total)
print("Largest branch:", branch_totals.index(max(branch_totals)) + 1)
print("Largest category:", category_totals.index(max(category_totals)) + 1)
print("Grand total:", sum(branch_totals))
```

8.Test Cases

Input/Test	Expected Result
Branch rows [1000,2000,1500], [3000,1000,2500], [1200,1800,2200]	Branch totals 4500, 6500, 5200; grand total 16200.
All values zero	All totals and averages are zero
Negative expense value	Should be rejected if input validation is added.

9.Sample Output Screenshot

```
Enter number of branches: 3
Enter number of expense categories: 4

Branch 1
Enter expense for category 1: 54
Enter expense for category 2: 10
Enter expense for category 3: 20
Enter expense for category 4: 30

Branch 2
Enter expense for category 1: 3
Enter expense for category 2: 6
Enter expense for category 3: 7
Enter expense for category 4: 8

Branch 3
Enter expense for category 1: 43
Enter expense for category 2: 56
Enter expense for category 3: 33
Enter expense for category 4: 67

--- Branch-wise Analysis ---
Branch 1 Total: ₹114.00
Branch 1 Average: ₹28.50
Branch 2 Total: ₹24.00
Branch 2 Average: ₹6.00
Branch 3 Total: ₹199.00
Branch 3 Average: ₹49.75

--- Category-wise Analysis ---
Category 1 Total: ₹100.00
Category 2 Total: ₹72.00
Category 3 Total: ₹60.00
Category 4 Total: ₹105.00

--- Largest Expenditure ---
Highest Spending Branch: Branch 3
Highest Spending Category: Category 4
Overall Expense: ₹337.00
```

10.Conclusion

The program summarises branch and category expenses and identifies the biggest contributors using a nested list.

55. Tuple-Based Transaction Audit

1. Problem Statement

Process transaction tuples containing an identifier, amount, and type; detect duplicate identifiers, invalid amounts, invalid types, and suspicious consecutive transactions.

2. Objectives

• Represent transactions as tuples.

• Detect duplicate transaction IDs.

• Flag zero or negative amounts and unknown transaction types.

• Flag unusually large consecutive transactions as suspicious.

3. Assumptions

• Each tuple contains (ID, amount, type).

• Valid types are credit and debit.

• Amounts must be greater than zero.

• For this classroom example, amounts above 10000 are flagged as unusually large; this is only a simple rule, not a fraud determination.

4. Algorithm

1. Start.

2. Create or input the transaction tuples.

3. Maintain a list of previously seen IDs.

4. Check each ID for duplicates.

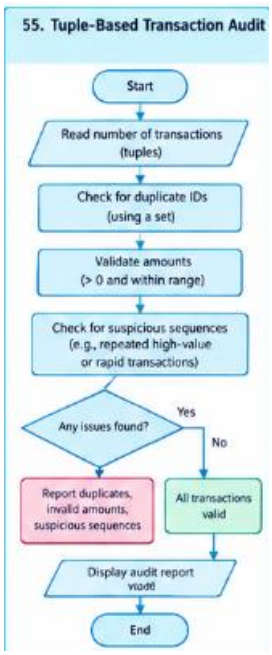
5. Validate amount and transaction type.

6. Flag amounts above the example threshold.

7. Display audit findings.

8. Stop.

5. Flowchart



6. Pseudocode

START

INPUT transaction tuples

seen = empty set

FOR each transaction (id, amount, type)

    IF id in seen: PRINT duplicate ID

    ELSE add id to seen

    IF amount <= 0: PRINT invalid amount

    IF type not credit or debit: PRINT invalid type

    IF amount > 10000: PRINT suspicious large amount

PRINT audit completed

STOP

7. Python Code

```
transactions = [
    (101, 500, "credit"),
    (102, 200, "debit"),
    (101, 700, "credit"),
    (103, -50, "debit"),
```

```
        (104, 15000, "transfer")
]
seen = set()
for tid, amount, kind in transactions:
        print("Transaction:", tid)
        if tid in seen:
            print("Duplicate ID")
        else:
            seen.add(tid)
        if amount <= 0:
            print("Invalid amount")
        if kind not in ("credit", "debit"):
            print("Invalid transaction type")
        if amount > 10000:
            print("Suspicious: unusually large amount")
print("Audit completed")
```

8.Test Cases

Input/Test	Expected Output
ID 101 appears twice	Second transaction with ID 101 is flagged as duplicate.
Amount -50	Invalid amount
Amount 15000	Flagged as unusually large under the example rule

9.Sample Output Screenshot

```
Transaction Audit Report

Transaction: 101 500 CREDIT

Transaction: 102 1000 DEBIT

Transaction: 101 700 CREDIT
Warning: Duplicate transaction ID
Alert: Suspicious transaction sequence

Transaction: 103 -200 DEBIT
Warning: Invalid transaction amount

Transaction: 104 5000 CREDIT
Alert: Suspicious transaction sequence

Transaction: 105 300 DEBIT

Audit Completed.
```

10.Conclusion

The audit checks transaction consistency and highlights records requiring manual review; its simple threshold is illustrative only.

56. Student Group Formation with Score Balancing

1. Problem Statement

Allocate students' scores into groups of a specified size while trying to balance the total score of each group.

2. Objectives

• Accept a list of student scores.

• Validate the required group size.

• Allocate each next student to the group with the lowest current total.

• Display each group's members and total score.

3. Assumptions

• Scores are non-negative integers

. • The group size must be positive.

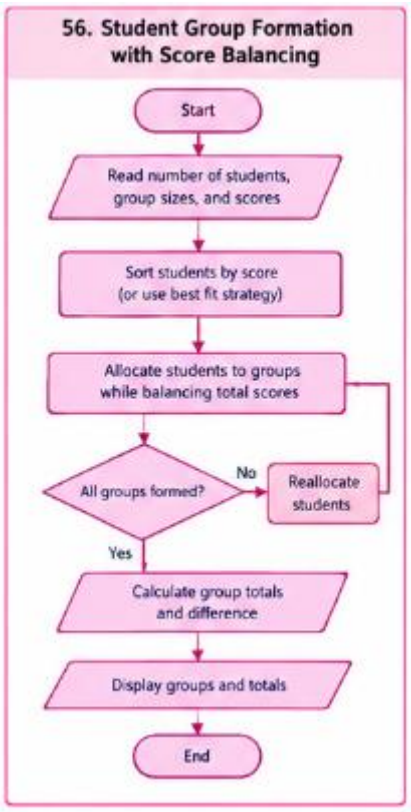
• The number of groups is rounded up if the students do not divide evenly.

• A greedy allocation is used; it aims to balance totals but does not guarantee a mathematically optimal solution.

4. Algorithm

1. Start.
2. Input student scores and group size.
3. Validate group size and scores.
4. Sort scores from highest to lowest.
5. Create enough empty groups to hold all students.
6. Assign each score to the group with the lowest current total.
7. Display groups and totals.
8. Stop.

5. Flowchart



6. Pseudocode

START

INPUT scores, group_size

IF group_size <= 0: PRINT error and STOP

SORT scores descending

CREATE ceil(number_of_scores / group_size) groups

FOR each score

CHOOSE group with lowest total that has fewer than group_size students

ADD score to group and update total

PRINT groups and totals

STOP

7. Python Code

```
scores = list(map(int, input("Enter scores separated by spaces: ").split()))
size = int(input("Students per group: "))
if size <= 0 or any(score < 0 for score in scores):
    print("Invalid group size or score")
else:
    count = (len(scores) + size - 1) // size
    groups = [[] for _ in range(count)]
    totals = [0] * count
    for score in sorted(scores, reverse=True):
        choices = [i for i in range(count) if len(groups[i]) < size]
        i = min(choices, key=lambda j: totals[j])
        groups[i].append(score)
        totals[i] += score
    for i in range(count):
        print("Group", i + 1, "scores:", groups[i])
        print("Total:", totals[i]
```

8.Test Cases

Input/Test	Expected Result
Scores 90 80 70 60; group size 2	Two groups are formed; greedy allocation balances totals
Group size 0	Invalid group size or score.
Scores 50 40 30; group size 2	Two groups; one group contains one student

9.Sample Output Screenshot

```
Enter student scores: 90 80 70 60
Enter group size: 2
Group 1 : [90, 60]
Total score: 150
Group 2 : [80, 70]
Total score: 150
Difference between highest and lowest group totals: 0
```

10.Conclusion

The program forms groups using a greedy balancing method and reports group membership and score totals.

57. Smart Canteen Order and Stock Management

1. Problem Statement

Process customer orders, calculate bills, update inventory, reject orders that exceed stock, and report daily sales and remaining stock.

2. Objectives

• Store item prices and stock quantities.

• Calculate the bill for each valid order.

• Prevent sales beyond available inventory.

• Summarise total sales and remaining stock.

3. Assumptions

• The sample menu contains idli, dosa, and juice.

• Prices and stock values are example values and can be changed.

• An order is accepted only if the item exists and quantity is positive and available.

• No tax or discount is included.

4. Algorithm

1. Start.

2. Create stock and price dictionaries.

3. Input the number of orders.

4. For each order, input item and quantity.

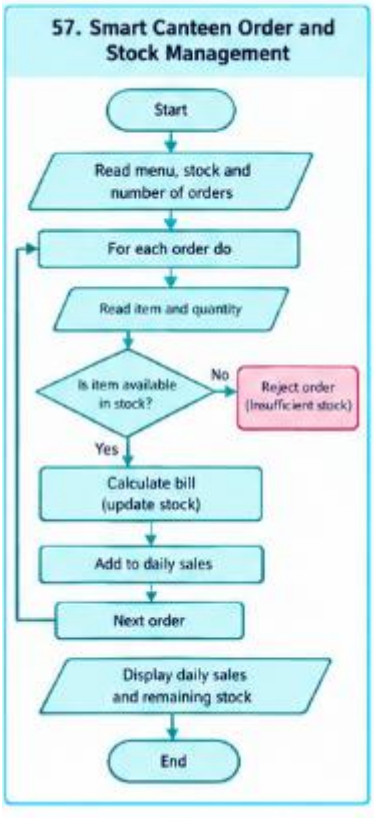
5. Validate item and quantity against stock.

6. If valid, calculate bill, reduce stock, and add to sales.

7. Display total sales and remaining stock.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
 SET stock and price dictionaries
 sales = 0
 INPUT number_of_orders
 REPEAT for each order
            INPUT item, quantity
            IF item exists AND 0 < quantity <= stock[item]
                    bill = price[item] * quantity
```

```
                stock[item] = stock[item] – quantity
                sales = sales + bill
                PRINT bill
            ELSE PRINT unavailable or invalid
PRINT sales and stock
STOP
```

7. Python Code

```
stock = {"idli": 20, "dosa": 15, "juice": 10}
price = {"idli": 20, "dosa": 40, "juice": 30}
sales = 0
orders = int(input("Number of orders: "))
for i in range(orders):
    item = input("Item (idli/dosa/juice): ").lower()
    qty = int(input("Quantity: "))
    if item in stock and 0 < qty <= stock[item]:
        bill = price[item] * qty
        stock[item] -= qty
        sales += bill
        print("Bill:", bill)
    else:
        print("Item unavailable or invalid quantity")
print("Daily sales:", sales)
print("Remaining stock:", stock)
```

8.Test cases

Input/Test	Expected Result
Order 2 dosa at 40 each	Bill 80; dosa stock reduces from 15 to 13.
Order 30 idli when stock is 20	Order rejected; stock unchanged
Unknown item pizza	Order rejected.

9.Output Screenshot

```
=== SMART CANTEEN ===

Menu:
Idly - Rs. 20 | Stock: 30
Dosa - Rs. 40 | Stock: 20
Meals - Rs. 80 | Stock: 15
Tea - Rs. 10 | Stock: 50
Juice - Rs. 30 | Stock: 25

Enter item name (or 'done'): dosa
Enter quantity: 3
Order successful!
Item: Dosa
Quantity: 3
Total Bill: Rs. 120

Menu:
Idly - Rs. 20 | Stock: 30
Dosa - Rs. 40 | Stock: 17
Meals - Rs. 80 | Stock: 15
Tea - Rs. 10 | Stock: 50
Juice - Rs. 30 | Stock: 25

Enter item name (or 'done'): idli
Invalid item!
```

10.Conclusion

The system processes valid orders, maintains inventory, rejects unavailable quantities, and totals daily sales

58. Library Borrowing and Fine Reconciliation

1. Problem Statement

Analyse borrowing records containing book IDs, borrowing duration, and return delay; calculate fines, identify overdue records, and estimate available books.

2. Objectives

• Process borrowing records stored as tuples.

• Calculate fines based on overdue days.

• Identify overdue books.

• Reconcile borrowed and available book counts.

3. Assumptions

• Each record contains (book ID, borrowed days, overdue days).

• Fine rate is 5 currency units per overdue day for this example.

• The sample library starts with 10 books and each record represents one distinct borrowed book.

• A record with zero overdue days has no fine.

4. Algorithm

1. Start.

2. Set fine rate and initial book count.

3. Read or define borrowing tuples.

4. For each record, validate day values.

5. Calculate fine = overdue days × fine rate.

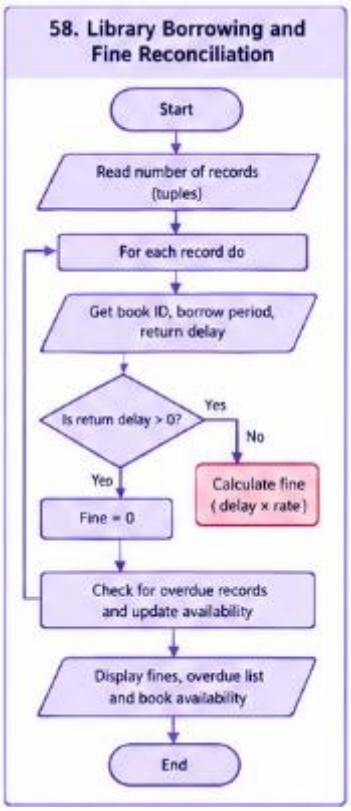
6. Report whether the record is overdue.

7. Subtract borrowed records from the available count.

8. Display total fine and availability.

9. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT records, initial_books, fine_rate
total_fine = 0
FOR each (book_id, borrowed_days, overdue_days)
        IF day values invalid: PRINT invalid record
        ELSE
                fine = overdue_days * fine_rate
                total_fine = total_fine + fine
                PRINT fine and overdue status
available = initial_books - number_of_valid_borrowed_books
PRINT total_fine, available
STOP
```

7. Python Code

```
records = [
        ("B101", 7, 3),
        ("B102", 5, 0),
         ("B103", 10, 4)
]
fine_rate = 5
total_books = 10
total_fine = 0
borrowed_count = 0
for book, borrowed_days, delay in records:
            if borrowed_days < 0 or delay < 0:
                print(book, "Invalid record")
                continue
            borrowed_count += 1
            fine = delay * fine_rate
            total_fine += fine
            print("Book:", book, "Fine:", fine)
            print("Overdue" if delay > 0 else "Returned on time")
 available = total_books - borrowed_count
print("Total fine:", total_fine)
print("Available books:", available)
```

8.Test Cases

Input/Test	Expected Result
B101, borrowed 7 days, delay 3	Fine = 15; overdue
B102, delay 0	Fine = 0; returned on time
Three valid records; initial stock 10	Available books = 7 while all three are borrowed.

9.Output Screenshot

```
LIBRARY FINE REPORT
--------------------------------------------
B101 - Returned on time, Fine: Rs. 0
B102 - Overdue by 3 days, Fine: Rs. 30
B103 - Overdue by 2 days, Fine: Rs. 20
B104 - Returned on time, Fine: Rs. 0
B105 - Overdue by 5 days, Fine: Rs. 50
--------------------------------------------
Total Fine: Rs. 100
Overdue Books: ['B102', 'B103', 'B105']
Books Returned on Time: 2
Total Books: 5
Books with Delayed Returns: 3
```

10.Conclusion

The program calculates overdue fines and reconciles available stock based on the provided borrowing records and example fine policy.

59. Campus Transport Allocation

1. Problem Statement

Estimate the buses needed for multiple routes using passenger demand and bus capacity, then identify passengers who cannot be served with the buses available.

2. Objectives

• Calculate required buses for each route.

• Account for capacity per bus.

• Calculate passengers served with the available buses.

• Report unserved passengers.

3. Assumptions

• Passenger demand is a non-negative integer.

• Each bus has a positive passenger capacity.

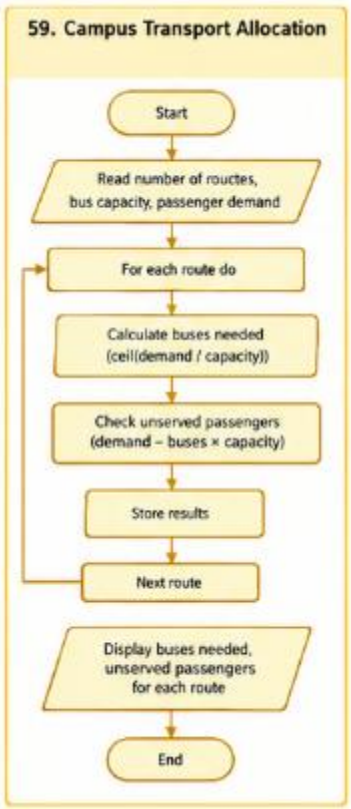
• Buses are fully usable for the route and no reserve capacity is modelled.

• Required buses = ceiling(passenger demand / bus capacity).

4. Algorithm

1. Start.
2. Input the number of routes.
3. For each route, input passenger demand, bus capacity, and available buses.
4. Validate all values.
5. Calculate required buses using ceiling division.
6. Calculate served passengers as the smaller of demand and available capacity.
7. Calculate unserved passengers.
8. Display results for each route.
9. Stop.

5. Flowchart



6. Pseudocode

```
START
 INPUT number_of_routes
 FOR each route
```

```
        INPUT demand, capacity, available_buses
        IF demand < 0 OR capacity <= 0 OR available_buses < 0
                PRINT invalid input
        ELSE
                needed = (demand + capacity - 1) // capacity
                served = MIN(demand, available_buses * capacity)
                unserved = demand - served
                PRINT needed, served, unserved
STOP
```

7. Python Code

```
routes = int(input("Number of routes: "))
for i in range(routes):
    print("Route", i + 1)
    demand = int(input("Passenger demand: "))
    capacity = int(input("Bus capacity: "))
    available = int(input("Buses available: "))
    if demand < 0 or capacity <= 0 or available < 0:
        print("Invalid input")
        continue
    needed = (demand + capacity - 1) // capacity
    served = min(demand, available * capacity)
    unserved = demand – served

    print("Buses needed:", needed)
    print("Passengers served:", served)
    print("Unserved passengers:", unserved)
```

8.Test Cases

Input/Test	Expected Result
Demand 95, capacity 40, available buses 3	3 buses needed; 95 served; 0 unserved

Demand 100, capacity 40, available buses 2	3 buses needed; 80 served; 20 unserved.
Capacity	Invalid input

9.Output Screenshot

```
Enter number of routes: 1
Enter bus capacity: 1
Enter passengers for route 1: 10

Route 1
Passengers: 10
Buses needed: 10
Unserved passengers: 0

Total buses needed: 10
Total unserved passengers: 0
```

10.Conclusion

The program estimates route-wise bus requirements and calculates unserved passengers when the available fleet is insufficient

60. Manufacturing Quality Control Matrix

1. Problem Statement

Analyse production quantities and defective units across products or batches, calculate defect percentages, and flag batches that exceed a quality threshold.

2. Objectives

• Represent batch information in a matrix.

• Validate production and defective quantities.

• Calculate defect percentage for each batch.

• Flag batches above the allowed defect percentage.

3. Assumptions

• Each row contains (units produced, defective units).

• Production quantity must be positive.

• Defective units must be between zero and units produced.

• A batch fails the check when its defect percentage is greater than the user-provided limit.

4. Algorithm

1. Start.

2. Create or input the production matrix and threshold.

3. For each batch, validate quantities.

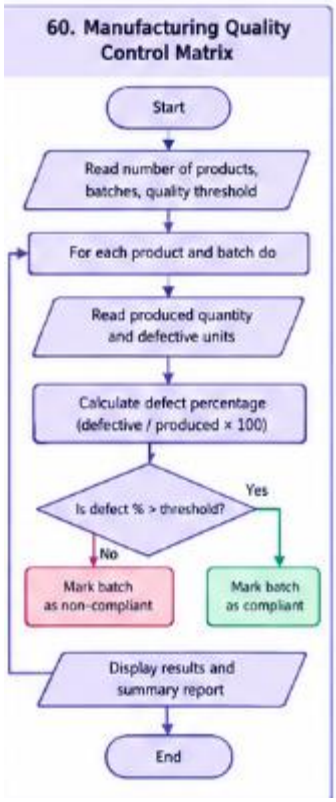
4. Calculate defect percentage = defective / produced × 100.

5. Compare with the quality threshold.

6. Display batch status and defect percentage.

7. Stop.

5. Flowchart



6. Pseudocode

START

INPUT defect_limit

FOR each batch (produced, defective)

```
        IF produced <= 0 OR defective < 0 OR defective > produced
            PRINT invalid batch
        ELSE
            percentage = defective / produced * 100
            PRINT percentage
            IF percentage > defect_limit: PRINT threshold exceeded
            ELSE: PRINT acceptable
STOP
```

7. Python Code

```
production = [
    [100, 5],
    [200, 20],
    [150, 6]
]
limit = float(input("Maximum defect percentage: "))
for i, (total, defective) in enumerate(production):
    if total <= 0 or defective < 0 or defective > total:
        print("Batch", i + 1, "Invalid data")
        continue
    percentage = defective / total * 100
    print("Batch", i + 1)
    print("Defect percentage:", round(percentage, 2), "%")
    if percentage > limit:
        print("Quality threshold exceeded")
    else:
        print("Quality acceptable")
```

8.Test Cases

Input/Test	Expected Result
100 produced, 5 defective, limit 5%	Defect percentage 5%; acceptable because it does not exceed the limit.

200 produced, 20 defective, limit 5%	Defect percentage 10%; threshold exceeded
Produced 100, defective 120	Produced 100, defective 12

9.Output Screenshot

```
Manufacturing Quality Control Report
------------------------------------

Product: Product A
Production Quantity: 1000
Defective Units: 20
Defect Percentage: 2.0 %
Status: Acceptable

Product: Product A
Production Quantity: 800
Defective Units: 50
Defect Percentage: 6.25 %
Status: Quality Alert

Product: Product B
Production Quantity: 1200
Defective Units: 30
Defect Percentage: 2.5 %
Status: Acceptable

Product: Product B
Production Quantity: 900
Defective Units: 100
Defect Percentage: 11.11 %
Status: Quality Alert

Product: Product C
Production Quantity: 700
Defective Units: 10
Defect Percentage: 1.43 %
Status: Acceptable

Quality check completed.
```

10.Conclusion

The program evaluates each batch against a defect-rate threshold and identifies batches that require quality review.